Assignment

Jnadi Ali

25 April 2026

The GitHub repository for this assignment is available at: `https://github.com/AliJnadi/RL_Assignment`

AI tools were used for code refinement and proofreading, but all implementation, experimentation, and analysis are my own. I am able to explain and justify all aspects of this work.

Due to hardware limitations (no access to a suitable GPU for high-fidelity simulators such as Isaac Sim), I used a CPU-based MuJoCo setup and, where necessary, a ROS-based fallback. This allowed me to complete all experiments in a reproducible manner.

Additional implementation details and reproduction instructions can be found in the repository README files.

1 Theory, Reward Design & PPO in Simulation

I trained a PPO agent as required and additionally evaluated SAC under the same reward formulations to better understand the effect of algorithm choice on learning with dense rewards.

I implemented and compared sparse and dense reward functions. While I did not implement scheduled shaping, I provide a detailed conceptual design and analysis of such a strategy.

1.1 Conceptual: RL Algorithms vs Classical Control

Classical methods, including proportional–integral–derivative (PID) control, inverse-kinematics (IK)-based position control, and model-predictive control (MPC), rely on explicit structure or models of the system. In contrast, modern RL algorithms such as Proximal Policy Optimization (PPO) and Soft Actor-Critic (SAC) learn policies directly from interaction data, typically without requiring an explicit dynamics model.

Classical control methods assume that the system dynamics are either known or can be approximated with sufficient accuracy. PID controllers are widely used for low-level joint control due to their simplicity, robustness, and real-time performance, although they require careful tuning and are limited in handling strong nonlinearities or coupling effects. IK-based control provides a geometric mapping from task space to joint space and is effective for structured positioning tasks, but may suffer from singularities and does not inherently account for system dynamics or contact interactions. MPC extends these approaches by incorporating a predictive model and solving an optimization problem over a finite horizon, allowing for constraint handling and improved performance in multi-variable systems. However, MPC depends heavily on model fidelity and can be computationally demanding, particularly in nonlinear settings.

Reinforcement learning methods remove the need for explicit modeling by learning control policies directly. PPO is an on-policy algorithm designed for stable policy updates, but typically requires a large number of samples and exhibits limited sample efficiency. SAC, an off-policy method, improves efficiency by reusing past experience through a replay buffer and promotes exploration via entropy maximization. This makes SAC particularly suitable for continuous control problems encountered in robotics. Both methods, however, assume a stationary environment during training and are generally trained in simulation due to the high data requirements.

A key practical distinction lies in exploration and safety. Classical control methods produce deterministic and predictable behavior, making them suitable for deployment on real hardware with strong safety requirements. In contrast, RL methods rely on stochastic exploration during training, which can lead to unsafe actions if applied directly on physical

systems. As a result, sim-to-real transfer techniques, such as domain randomization and system identification, are typically required before deployment.

In practice, classical control methods are preferred when a reliable model or task structure is available, when safety constraints are strict, or when data collection on physical hardware is costly. For example, industrial pick-and-place operations using fixed objects and known geometries are typically implemented using IK-based trajectory generation combined with PID joint control, as the task can be solved deterministically with high precision and repeatability. Similarly, tasks such as robotic welding or CNC machining rely on precomputed trajectories and model-based control to guarantee accuracy and avoid unsafe behavior, making learning-based exploration undesirable.

Rather than training policies from scratch, a common engineering approach is to combine classical control with learning-based components in a structured manner. In robotic grasping, for instance, an IK-based controller can be used to move the end-effector to a candidate grasp pose computed from perception, while a reinforcement learning policy adjusts finger forces or local positioning to account for object uncertainty and contact dynamics. In peg-in-hole assembly, a nominal trajectory can be generated using classical planning, while a learned residual policy compensates for small misalignments and frictional effects that are difficult to model analytically. Similarly, in force-sensitive tasks such as surface polishing or insertion, classical impedance or position control ensures stability, while a learning-based component refines the interaction strategy.

This hybrid paradigm, often referred to as residual reinforcement learning, improves sample efficiency by restricting learning to a lower-dimensional correction space and significantly reduces the risks associated with unconstrained exploration, making it more suitable for deployment on real robotic systems. Table. 1 shows a summary comparisons.

Table 1: Comparison of classical control methods and reinforcement learning algorithms in robotic manipulation.

Aspect	**PID / IK / MPC**	**PPO**	**SAC**
Core assumptions	Requires known or approximated dynamics; structured system	Model-free; stationary environment; on-policy updates	Model-free; stationary environment; off-policy learning
Sample efficiency	Very high (no training required)	Low; requires many interactions	Moderate; improved via replay buffer
Wall-clock efficiency	Real-time for PID/IK; MPC may be expensive	Slow training; fast inference	Faster training than PPO; fast inference
Exploration behavior	Deterministic; no exploration	Stochastic policy	Stochastic policy with entropy maximization
Stability and safety	Strong guarantees; suitable for hardware	Limited guarantees; unsafe during training	Improved over PPO but still limited guarantees
Robustness to model errors	Limited (except robust MPC variants)	High	High
Suitability for contact-rich tasks	Limited due to modeling difficulty	Possible but data inefficient	Better suited due to efficient learning
When not to use	Unknown or highly complex dynamics	Data-limited or safety-critical settings	Strict safety-critical settings without safeguards

For contact-rich manipulation tasks involving deformable objects, purely model-based approaches are generally insufficient due to the difficulty of accurately modeling nonlinear and discontinuous interactions. In this setting, a hybrid architecture is recommended. Classical controllers should be used at the low level to ensure stable and safe execution, while a model-free RL algorithm is used at a higher level to learn interaction strategies. Among RL methods, SAC is a more appropriate choice than PPO due to its higher sample efficiency and better exploration properties in continuous action spaces. Combined with simulation-based training and domain randomization, this approach provides a practical and robust solution for deformable-object manipulation.

1.2 Conceptual: Reward Shaping for Humanoid Manipulation

Considering a dual-arm humanoid manipulator tasked with grasping a deformable object and placing it at a target pose, I define the reward at time t as a weighted combination of shaping terms and penalties:

$$r_t = \sum_i w_i r_t^{(i)} - \sum_j \lambda_j c_t^{(j)}, \tag{1}$$

where $w_i \geq 0$ and $\lambda_j \geq 0$ are scalar weights. This formulation follows standard reward decomposition used in robotic manipulation, where shaping terms guide behaviour while penalties enforce safety and regularisation [1, 2].

1.2.1 Reward Decomposition

I design the reward to reflect the natural stages of manipulation: reaching, aligning, contacting, grasping, and placing, consistent with common structured manipulation pipelines [3].

The reaching term

$$r^{\text{reach}} = -\|\mathbf{p}_{\text{ee}} - \mathbf{p}_{\text{grasp}}\|_2, \tag{2}$$

is a dense signal based on proprioception and perception. It encourages approach behaviour but may lead to premature or unsafe contact if not gated.

To ensure correct orientation, I include an alignment term

$$r^{\text{align}} = \exp(-\alpha \angle(\mathbf{R}_{\text{ee}}, \mathbf{R}_{\text{target}})), \tag{3}$$

which depends on visual estimation of object geometry. Such geometric shaping is widely used but can introduce local minima, particularly in symmetric objects.

For physical interaction, I define a contact-based reward

$$r^{\text{contact}} = \mathbf{1}_{\text{contact}} \cdot \sum_i \min(F_{n,i}, F_{\max}), \tag{4}$$

which relies on tactile sensing. Visuo-tactile feedback has been shown to be critical for deformable object manipulation, as it provides information unavailable from vision alone [4, 5].

To guide task completion, I use a dense object-to-goal term

$$r^{\text{obj}} = -\|\mathbf{p}_{\text{obj}} - \mathbf{p}_{\text{goal}}\|_2, \tag{5}$$

which requires object tracking (e.g. keypoints or point clouds). While effective, it can lead to pushing behaviours if activated before a stable grasp.

I also include a grasp quality signal based on caging:

$$r^{\text{cage}} = \text{IoU}(\mathcal{V}_{\text{gripper}}, \mathcal{V}_{\text{object}}), \tag{6}$$

which measures geometric enclosure and is motivated by prior work on caging-based grasping for deformable objects [6].

The penalty terms regularise behaviour. Energy usage is penalised via

$$c^{\text{energy}} = \|\boldsymbol{\tau}\|_2^2, \tag{7}$$

while joint limits and self-collisions are handled through kinematic constraints:

$$c^{\text{joint}} = \sum_k \max(0, \theta_k - \theta_k^{\max}) + \max(0, \theta_k^{\min} - \theta_k), \tag{8}$$

$$c^{\text{coll}} = \sum_{\text{pairs}} \max(0, d_{\text{safe}} - d_{ij}). \tag{9}$$

To encourage smooth control, I include a jerk penalty

$$c^{\text{jerk}} = \|\dddot{\boldsymbol{\theta}}\|_2^2, \tag{10}$$

although excessive weighting can reduce task efficiency.

Finally, I penalise excessive deformation:

$$c^{\text{deform}} = \max(0, E_{\text{strain}} - \epsilon) + \mathbf{1}_{F_{n,i} > F_{\text{safe}}}, \tag{11}$$

which is particularly important in deformable object manipulation, where maintaining structural integrity is essential [2].

1.2.2 Potential-Based Shaping and Policy Invariance

Potential-based shaping preserves optimal policies when rewards take the form

$$F(s, s') = \gamma\Phi(s') - \Phi(s), \tag{12}$$

as shown by Ng et al. [7]. In my formulation, the object-to-goal term can be interpreted as a potential function when the goal is static. However, since I use it directly rather than as a difference, it introduces bias.

Other terms, such as reaching and alignment, are not potential-based due to their dependence on dynamic targets (e.g. deformable grasp points). Therefore, the overall reward is not policy-invariant.

I accept this bias because sparse rewards are ineffective in high-dimensional manipulation tasks. Techniques such as Hindsight Experience Replay further demonstrate the importance of shaping for improving sample efficiency in sparse settings [8].

1.2.3 Deformable-Specific Challenges

Manipulating deformable objects introduces additional challenges compared to rigid-body tasks [9, 2]. First, there is no single well-defined pose, as the object has effectively infinite degrees of freedom. I address this by tracking multiple keypoints or surface features.

Second, the internal state (e.g. strain or stress) is only partially observable, making the problem inherently a Partially Observable Markov Decision Process (POMDP). Tactile sensing and temporal information are therefore essential for capturing hidden dynamics.

Third, success criteria are ambiguous. For example, placing a cloth requires satisfying geometric alignment, force constraints, and structural integrity. I define success as a combination of these metrics.

Finally, deformable objects exhibit complex contact dynamics that are difficult to simulate accurately. This creates a significant sim-to-real gap [10], which motivates the use of domain randomization and robust sensing modalities.

1.2.4 Curriculum and Scheduling Strategy

To stabilise learning and mitigate the limitations of dense reward shaping, I suggest using a stage-gated curriculum that progressively shifts the objective from dense guidance to sparse task completion, similar in spirit to curriculum learning strategies used in complex robotic tasks [11].

The training progresses through reaching, grasping/lifting, and placing phases, with transitions triggered by success-rate thresholds. Early stages focus on geometric positioning, while later stages emphasise contact-rich manipulation and final task completion. Shaping terms are gradually annealed, and a sparse success bonus is introduced to align the final policy with the true objective.

In parallel, it is better to add automatic domain randomization (ADR), where environment parameters such as stiffness, friction, and initial conditions are adapted based on performance. This creates an adaptive curriculum that improves robustness and generalisation.

Algorithm 1 Stage-Gated Curriculum with ADR

Initialize weights: $w_{\text{reach}} = 1, w_{\text{align}} = 1$, others 0; λ_j small constants.
Phase $\leftarrow$ `REACH`; ADR $\sigma \leftarrow 0$ (or small)
while not converged **do**
  Sample environment with physical parameters $\phi \sim \mathcal{N}(\phi_{\text{nom}}, \sigma^2)$
  Execute episode with current reward weights, collect trajectories
  Update success counters for current phase condition
  if Phase == `REACH` and mean_success(reach_2cm) > 0.8 **then**
    Phase $\leftarrow$ `GRASP`; $T_1 \leftarrow$ current steps
    $w_{\text{contact}} \leftarrow 1.0$, $w_{\text{cage}} \leftarrow 0.5$, $w_{\text{obj}} \leftarrow 0.5$
  else if Phase == `GRASP` and mean_success(lift_5cm) > 0.7 **then**
    Phase $\leftarrow$ `PLACE`; $T_2 \leftarrow$ current steps
    $w_{\text{obj}} \leftarrow 2.0$
  end if
  if Phase == `GRASP` **then**
    $w_{\text{reach}} \leftarrow \max(0, 1 - (t - T_1)/200000)$
    $w_{\text{align}} \leftarrow \max(0, 1 - (t - T_1)/200000)$
  else if Phase == `PLACE` **then**
    $w_i \leftarrow w_i \cdot 0.99$ (for all shaping i)
    $w_{\text{success}} \leftarrow \min(1, w_{\text{success}} + 0.001)$ ▷ ramp sparse bonus
  end if
  ADR update: If recent success > 0.9, $\sigma \leftarrow \sigma + 0.01$, else $\sigma \leftarrow \max(0, \sigma - 0.005)$
  Update policy with experience batch
end while

Overall, this curriculum balances guidance and objective fidelity: dense rewards accelerate early learning, while annealing ensures that the final behaviour reflects the true task definition.

1.2.5 Evaluation Protocol

To evaluate policy quality, I consider going beyond episode return and consider task-specific metrics. The primary metric is success rate over 100 evaluation episodes, measured under both deterministic and stochastic policies. I also consider time-to-grasp and force profiles, which capture efficiency and safety.

For deformable objects, we need additionally to evaluate generalisation to unseen shapes and material properties, as well as qualitative behaviour such as smoothness and stability. This is consistent with recent work on dexterous manipulation, where robustness and generalisation are critical performance indicators [12].

Task success is defined using a combination of geometric accuracy, safe force limits, and preservation of object structure, ensuring that the learned policy is both effective and physically plausible.

1.3 Practical: PPO Training & Reward Ablation

I trained a PPO agent on the `FetchPickAndPlace-v4` task (MuJoCo), comparing different reward formulations. In particular, I implemented a sparse reward and a dense shaped reward, and additionally experimented with SAC using the same dense reward when PPO failed to learn effectively.

The sparse reward is defined as a binary signal, where the agent receives a reward only upon successful completion of the task. In contrast, the dense reward is designed to guide the agent throughout the episode by decomposing the task into intermediate objectives. Specifically, the dense reward combines (i) a reaching term that minimizes the distance between the end-effector and the object, (ii) a gripper shaping term that encourages opening when far and closing when near the object, (iii) a lifting component consisting of both a continuous shaping term and a one-time bonus when the object is lifted above a threshold, (iv) a placing term that minimizes the distance between the object and the goal, and (v) a small action penalty to regularize control. A final success bonus is granted when the object reaches the target location. This shaping is intended to decompose the long-horizon manipulation problem into smoother subgoals while still preserving the final objective.

Figure 1 shows the mean episode return, and Figure 2 shows the task success rate, both averaged across seeds with shaded ±1 standard deviation.

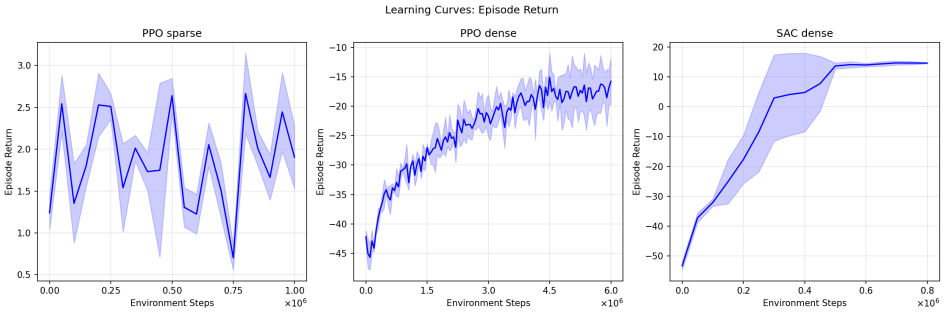

Figure 1: Learning curves for sparse reward (PPO), dense reward (PPO), and dense reward with SAC.

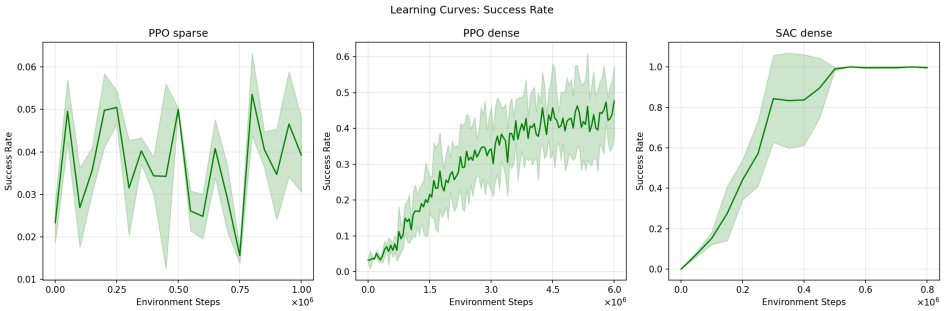

Figure 2: Success rate for sparse reward (PPO), dense reward (PPO), and dense reward with SAC.

From the results, PPO with sparse reward learns extremely slowly. Even after 1M environment steps, the episode return remains close to zero and the success rate barely exceeds 5%. The learned behaviour resembles a random policy, which is expected since the reward signal is too infrequent to provide meaningful gradients for exploration in such a long-horizon task.

Using the dense reward improves learning slightly, as the agent now receives continuous feedback encouraging it to approach, grasp, and lift the object. However, PPO still struggles to solve the task. The success rate reaches only around 40%, and the behaviour often converges to suboptimal strategies such as hovering near the object or pushing it without successfully grasping it. This suggests that although the shaping provides useful local guidance, it also introduces local optima that are difficult to escape in an on-policy setting.

In contrast, SAC with the same dense reward learns much more effectively, reaching approximately 40% success within 200k steps. The resulting behaviour is smooth and consistent, indicating that the replay buffer and off-policy updates significantly improve sample efficiency and stability when learning from dense signals.

After training, each policy was evaluated over 100 episodes under both deterministic and stochastic action selection. The results are summarised in Table 2.

Experiment	**Deterministic SR**	**Stochastic SR**	**Seeds**
PPO sparse (A)	0.04 ± 0.01	0.03 ± 0.03	3
PPO dense (B)	0.15 ± 0.05	0.09 ± 0.00	3
SAC dense (B)	0.89 ± 0.04	0.83 ± 0.06	3

Table 2: Final success rates (100 evaluation episodes) for each variant.

Overall, only the dense reward combined with SAC produces a reliable policy, while both PPO variants fail to fully solve the task. The sparse reward is too uninformative, and although the dense reward introduces helpful structure, PPO appears unable to effectively exploit it due to high variance and limited sample reuse.

Regarding reward design, the reaching and placing terms were essential for guiding the agent toward the object and goal, while the lifting bonus was particularly important for

overcoming the sparse transition between grasping and successful transport. However, the gripper shaping term sometimes encouraged premature closing or suboptimal contact strategies, contributing to local minima.

No severe reward hacking was observed, although PPO with dense reward occasionally learned to "flick" the object off the table. In this case, the distance-to-goal term decreased temporarily without achieving a valid lift-and-place behaviour. This issue was partially mitigated by conditioning the reward on the object being above a minimum height before applying the goal-distance term.

I did not implement a full scheduling strategy (variant C). Instead, I interpret the comparison between PPO and SAC as highlighting the importance of optimisation dynamics. Even with identical reward shaping, the terminal behaviour differs significantly, with PPO producing hesitant and unstable motions, while SAC yields smooth and reliable trajectories.

Finally, I briefly examined sensitivity to key PPO hyperparameters using the dense reward. Increasing the entropy coefficient improved early exploration but had little effect on final performance. Adjusting GAE λ from 0.95 to 0.98 produced no noticeable change. Increasing the learning rate to 3×10^{-4} led to premature convergence to poor policies, indicating instability. Overall, PPO in this setting is highly sensitive to the exploration–exploitation balance and struggles to escape local optima, even with carefully designed reward shaping.

2 Part 2 (ROS2 Fallback)

I chose this path because I have a MacBook Air M1 and I am unable to run any tasks that require a local GPU. In the GitHub repository you can find the ROS2 package, including the launch file, as well as two rosbag recordings: one for a successful run and one for a failed run. To obtain the failed run, I added noise to the observation, causing the agent to fail the task. I used an SAC model because it gave better results than PPO for the same reward function.

The system architecture is shown in Fig. 3, which was generated using `rqt_graph`.

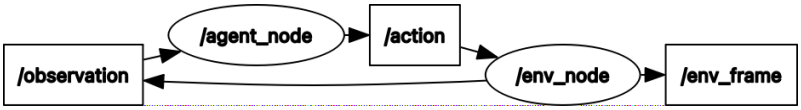

Figure 3: ROS2 node graph showing the simulator node, policy node, and the topics used for communication.

I ran the system for 100 episodes and obtained the following results:

- Control-loop latency: mean = 8.23 ms, 95th percentile = 14.07 ms.
- Policy inference time: 0.99 ms.
- Success rate: 93%.

References

[1] A. Billard and D. Kragic, "Trends and challenges in robot manipulation," *Science*, vol. 364, no. 6446, p. eaat8414, 2019.

[2] Y. Li, X. Wang, *et al.*, "A survey on robotic manipulation of deformable objects: Recent advances, open challenges and new frontiers," *IEEE Transactions on Robotics*, 2023.

[3] Y. Zhou, J. Chen, *et al.*, "Grasping deformable objects in industry application: A comprehensive review of robotic manipulation," *Applied Sciences*, 2021.

[4] Z. Chen, H. Liu, *et al.*, "Grasping deformable objects via reinforcement learning with cross-modal attention to visuo-tactile inputs," *IEEE Robotics and Automation Letters*, 2022.

[5] Y. Lee, S. Hong, M.-g. Kim, G. Kim, and C. Nam, "Grasping deformable objects via reinforcement learning with cross-modal attention to visuo-tactile inputs," *arXiv preprint arXiv:2504.15595*, 2025.

[6] A. Rodriguez and M. T. Mason, "Caging-based grasping of deformable objects for geometry-based robotic manipulation," *The International Journal of Robotics Research*, 2018.

[7] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in *Proceedings of the 16th International Conference on Machine Learning (ICML)*, 1999.

[8] M. Andrychowicz, F. Wolski, A. Ray, J. Schneider, R. Fong, P. Welinder, B. McGrew, J. Tobin, O. Pieter Abbeel, and W. Zaremba, "Hindsight experience replay," *Advances in neural information processing systems*, vol. 30, 2017.

[9] J. Sanchez, J.-A. Corrales, B.-C. Bouzgarrou, and Y. Mezouar, "Robotic manipulation and sensing of deformable objects in domestic and industrial applications: a survey," *The International Journal of Robotics Research*, vol. 37, no. 7, pp. 688–716, 2018.

[10] S. Ibrahim, M. Mostafa, A. Jnadi, H. Salloum, and P. Osinenko, "Comprehensive overview of reward engineering and shaping in advancing reinforcement learning applications," *IEEE Access*, vol. 12, pp. 175473–175500, 2024.

[11] I. Akkaya, M. Andrychowicz, M. Chociej, M. Litwin, B. McGrew, A. Petron, A. Paino, M. Plappert, G. Powell, R. Ribas, *et al.*, "Solving rubik's cube with a robot hand," *arXiv preprint arXiv:1910.07113*, 2019.

[12] Y. Qin, S. Li, *et al.*, "Universal dexterous functional grasping via demonstration-editing reinforcement learning," *Science Robotics*, 2023.